

Autoévaluation

Améliorez et ajoutez des fonctionnalités



Un dernier doute avant l'envoi de vos livrables ?

Pour vérifier la qualité de votre travail :

- cochez les cases ci-dessous : elles indiquent que vous avez bien pris en compte chaque indicateur de réussite ;
- renseignez, si besoin, la colonne "Notes" avec des commentaires sur vos livrables / vos étapes. Ils seront des points de discussion avec votre mentor pendant votre session de bilan / soutenance.

Quand toutes les cases de ce document seront cochées, vous pourrez déposer vos livrables sur la plateforme.

Bonne réussite !

Étape	Indicateurs de réussite de l'activité	Notes
<i>Étape 1 – Analysez le code existant</i>	■ J'ai installé toutes les technologies nécessaires : Java, Angular, Docker ■ J'ai réussi à lancer le back-end et le front-end ■ L'application fonctionne correctement en créant un utilisateur agent. ■ J'ai exploré la structure du projet (dossiers, fichiers, logique générale). ■ J'ai pris des notes ou fait un schéma pour mieux comprendre l'architecture full-stack. ■ Je peux expliquer la structure et le fonctionnement du code existant à orale (par ex. lors d'une session de mentorat).	Installation de : Back-end - Java (openjdk 21.0.11) - Maven (3.9.12) - Docker Desktop - STS (Spring Tools 5.2.0) - Postman (12.18.5) Front-end - Angular (19.2.15) - VS Code (1.128.0) Architecture : > controller > service > repository > entities > dto > mapper > configuration > security > EtudiantBackendApplication
<i>Étape 2 – Corrigez l'API d'authentification des utilisateurs côté back-end</i>	■ J'ai analysé le code existant avant de faire toute modification. ■ J'ai identifié les fichiers contenant des bugs ■ J'ai noté les comportements inattendus ou les erreurs visibles. ■ J'ai apporté les corrections nécessaires pour que l'API /api/login fonctionne. Désormais, elle retourne un token JWT. ■ J'ai testé cette API avec succès sur Postman.	> Lancement de l'application en mode Debug > On place des breakpoints dans UserController, UserService et JwtService > On envoie une requête POST /api/login depuis Postman > On vérifie que LoginRequestDTO contient les login et MDP > Contrôle d'appel à userRepository.findByLogin(login) et récupération de l'utilisateur > Matching du MDP avec passwordEncoder.matches() > On suit l'exécution jusqu'à JwtService.generateToken() > Contrôle de la création et de la signature du token JWT > Vérification dans Postman > code HTTP 200

<i>Étape 3 – Implémentez l'interface d'authentification côté front-end</i>	■ J'ai implémenté un écran d'authentification simple avec login et mot de passe. ■ J'ai connecté le nouvel écran à la route d'authentification ■ J'ai implémenté un service Angular pour appeler l'API. ■ J'ai vérifié la sécurisation de l'écran. ■ J'ai testé la fonctionnalité du nouvel écran. ■ J'ai géré les erreurs serveur (ex. mauvais identifiants) et les états (chargement, erreur, succès).	Construction d'un écran de connexion comportant un champ Login et mot de passe accessible via la route /login Le service Angular envoie les identifiants à l'API REST du back-and (POST /api/login) Si auth réussie, le serveur renvoie un token JWT enregistré dans le localStorage qui est ajouté à toutes mes requêtes HTTP => HTTP Interceptor
<i>Étape 4 – Ajoutez de nouvelles fonctionnalités côté back-end</i>	■ J'ai implémenté les routes nécessaires pour gérer les étudiants (fonctions CRUD) : ■ ajouter un étudiant ■ consulter la liste des étudiants et les informations détaillées d'un étudiant ■ modifier un étudiant ■ supprimer un étudiant ■ Toutes les routes de l'API (existantes et nouvelles) fonctionnent correctement. ■ J'ai vérifié la sécurisation des routes pour assurer l'accès uniquement aux utilisateurs authentifiés. ■ J'ai testé chaque route avec succès sur Postman. ■ J'ai respecté les bonnes pratiques de développement back-end en Java, notamment le découpage du code en suivant le modèle en couche.	Implémentation de l'ensemble des opérations CRUD : > Création, consultation de la liste et du détail d'un étudiant, modification et suppression d'un étudiant > Une fonctionnalité correspond à une route REST (GET, POST, PUT, DELETE) développées avec Spring Boot > Architecture en couches (best practice) avec respect des responsabilités (Controller, Service, Repository, DTO, Mappers) > Les routes ont été sécurisées (Spring Security) et JWT > Test des endpoints avec Postman

Étape 5 –
Implémentez les
écrans côté
front-end

- J'ai implémenté les écrans nécessaires pour gérer les étudiants (en consommant les API CRUD du back-end) :
 - **ajouter** un étudiant
 - **consulter** la liste des étudiants et les informations détaillées d'un étudiant
 - **modifier** un étudiant
 - **supprimer** un étudiant
- J'ai vérifié la sécurisation des écrans pour assurer l'accès uniquement aux utilisateurs authentifiés.
- J'ai testé tous les écrans et vérifié leur fonctionnement côté l'utilisateur.
- J'ai respecté les bonnes pratiques de développement front-end en Angular, notamment l'utilisation des services Angular.

(Pour ce projet, il n'a pas besoin de respecter de règles particulières concernant l'apparence visuelle ou l'accessibilité.)

> Possibilité d'ajouter un étudiant, d'afficher la liste des étudiants, de consulter le détail d'un étudiant, de modifier ses informations et de le supprimer
> Les services Angular permettent de communiquer avec le Back-end (appels HTTP > différentes API)
> Auth Guard vérifie qu'un utilisateur est authentifié afin d'autoriser la navigation vers les routes sécurisées
> HTTP Interceptor ajoute le token JWT à chaque requête envoyée vers le bak-end
> Test de l'ensemble des fonctionnalités (opérations CRUD, communication avec les API, gestion des erreurs, navigation entre les différents écrans)

Autoévaluation

Effectuez des tests unitaires et d'intégration



Un dernier doute avant l'envoi de vos livrables ?

Pour vérifier la qualité de votre travail :

- cochez les cases ci-dessous : elles indiquent que vous avez bien pris en compte chaque indicateur de réussite ;
- renseignez, si besoin, la colonne "Notes" avec des commentaires sur vos livrables / vos étapes. Ils seront des points de discussion avec votre mentor pendant votre session de bilan / soutenance.

Quand toutes les cases de ce document seront cochées, vous pourrez déposer vos livrables sur la plateforme.

Bonne réussite !

Étape	Indicateurs de réussite de l'activité	Notes
<i>Étape 1 – Analysez le code de test fourni dans le back-end</i>	■ J'ai relu les fichiers de tests déjà présents dans le back-end. ■ J'ai identifié les classes, services et controllers déjà testés. ■ J'ai compris les logiques de test utilisées dans ces fichiers. ■ J'ai exécuté les tests fournis pour vérifier leur comportement. ■ J'ai pris des notes sur les bonnes pratiques à réutiliser pour mes futurs tests.	La couche UserControllerTest.java renvoie un test d'intégration et simule un comportement Spring et simulant le comportement de l'endpoint : <pre>> registerUserWithoutRequiredData() > registerAlreadyExistUser() > registerUserSuccessful()</pre> La couche UserServiceTest.java renvoie des tests unitaires. @Mock crée de faux Repository et @InjectMocks utilise UserService mais en lui injectant de faux objets.
<i>Étape 2 – Écrivez un plan de tests</i>	■ J'ai listé les cas de tests nécessaires pour valider le fonctionnement du back-end, du front-end et du bout au bout. ■ J'ai ajouté des tests unitaires pour les composants et les services. ■ J'ai organisé mes cas de test par priorité (simple → complexe). ■ J'ai distingué les entrées attendues et les sorties attendues pour chaque test.	Tests par niveau de complexité : Tests unitaires <pre>> Vérification des règles métier du UserService > Vérification de la validité des données > Vérification des exceptions</pre> Tests d'intégration <pre>> Vérification des endpoints REST > Vérification du code HTTP > Vérification des interactions entre les couches Controller, Service, Repository Tests End-to-End > Vérification du parcours complet d'un (authentification, liste, ajout, modification, suppression, contrôle de la sécurité) utilisateur</pre>
<i>Étape 3 – Implémentez les tests unitaires et d'intégration du back-end</i>	■ J'ai lu les documentations sur JUnit et sur Mockito pour monter en compétence sur ces technologies. ■ J'ai écrit des tests unitaires pour les nouveaux services du back-end. ■ J'ai écrit des tests d'intégration pour les nouveaux controllers.	

	■ J'ai documenté mes tests avec des commentaires utiles. ■ J'ai atteint au moins 80 % de couverture de test sur le back-end. ■ J'ai généré de rapports de couverture de test indiquant le seuil de 80 % minimum.	
<i>Étape 4 – Implémentez les tests unitaires et d'intégration du front-end</i>	■ J'ai lu les documentations sur Jest pour monter en compétence sur cette technologie. ■ J'ai écrit des tests unitaires pour les services Angular et pour les composants Angular. ■ J'ai vérifié que tous les tests s'exécutent avec succès. ■ J'ai atteint au moins 80 % de couverture de test sur le front-end avec Jest. ■ J'ai généré de rapports de couverture de test indiquant le seuil de 80 % minimum.	
<i>Étape 5 – Implémentez les tests E2E du front-end</i>	■ J'ai lu les documentations sur Cypress pour monter en compétence sur cette technologie. ■ J'ai écrit des tests E2E pour les écrans principaux de l'application. ■ J'ai commencé par les formulaires simples (inscription, connexion). ■ J'ai vérifié que tous les tests E2E passent sans erreur. ■ J'ai atteint au moins 80 % de couverture E2E sur les parcours utilisateurs. ■ J'ai généré de rapports de couverture de test indiquant le seuil de 80 % minimum.	